

STEPS AND ALGORITHM

AL ML-Based Project on Movie Recommendation System

*-Tanisha Rawat(25SCS1003005544), Gaurika Dubey(25SCS1003005438),
Ayush Sharma(25SCS1003005460), Charvi Sharma(25SCS1003005391)*

1. Steps:

1.1. Data Collection & Preprocessing

- Downloaded MovieLens 100K dataset
- Loaded u.item file containing movie information (movie_id, title, genres)
- Loaded u.data file containing user ratings (user_id, movie_id, rating, timestamp)
- Cleaned and structured data for analysis

1.2. System Architecture Design

Implemented 5 recommendation approaches:

- Collaborative Filtering (User-Based)
- Content-Based Filtering (Movie Search)
- Genre-Based Filtering
- Popularity-Based (Trending)
- Hybrid System

1.3. Algorithm Implementation

- Developed individual recommendation modules
- Integrated all modules into a unified system
- Added user interaction interface

1.4. Testing & Validation

- Tested with various user inputs
- Validated recommendation quality
- Ensured system robustness

2. Core Algorithms & Techniques Used:

2.1. Collaborative Filtering (User-Based)

➤ **Main Algorithm: k-Nearest Neighbors (k-NN)**

It's like finding your "movie soulmates" - people who have similar tastes to you.
Example: If someone loves Avengers, Star Wars, and Jurassic Park, k-NN finds other users who also love these same movies, then it recommends other movies.

➤ **Similarity Metric: Cosine Similarity or Pearson Correlation (for finding similar users)**

It is a way to measure "how similar" two things are.

- **Cosine Similarity:** Measures the angle between rating patterns.
E.g., User A ratings: [5, 4, 3] for [Avengers, Titanic, Horror Movie]
User B ratings: [5, 4, 3] for same movies → Arrows point same direction = HIGH similarity
User C ratings: [1, 2, 1] for same movies → Arrows point in different directions = LOW similarity
- **Pearson Correlation:** Measures how ratings move together
E.g., When User A gives a movie 5 stars, does User B also give it 5 stars?
When User A hates a movie (1 star), does User B also hate it?
If YES → High correlation = Similar tastes

2.2. Content-Based Filtering

A. Genre-Based Recommendations:

- Algorithm: Content-Based Filtering using categorical features
- Method: Filtering and ranking movies by genre

B. Movie Search & Similarity:

- Text Processing: Fuzzy String Matching (for "Avenger" search)
- Algorithm: Content-Based Filtering based on movie metadata

2.3. Hybrid Recommendation System

- Combination Method: Weighted Hybrid Approach
- Technique: Blending Collaborative + Content-based recommendations

2.4. Popularity-Based Filtering (Trending Movies)

- Algorithm: Ranking Algorithm with scoring formula
- Scoring Method: Likely uses Bayesian Average or Weighted Rating

2.5. Data Processing & Feature Engineering

- Rating Normalization: Handling 1–5-star ratings

- Genre Vectorization: Converting genre strings to feature vectors
- User-Item Matrix: Building sparse matrices for collaborative filtering

3. Primary ML Algorithms:

- k-Nearest Neighbors (k-NN) for user-based collaborative filtering
- Cosine Similarity for measuring user/movie similarity
- Content-Based Filtering using genre and metadata features
- Hybrid Recommendation System: Combining multiple approaches
- Bayesian Average/Weighted Rating for popularity ranking

4. Supporting Techniques:

- Fuzzy String Matching for movie search functionality
- Text Processing for movie metadata handling
- Matrix Operations for user-item interactions
- Data Normalization for rating standardization

5. Key Features Implemented:

- User-based collaborative recommendations
- Genre-based content filtering
- Movie search and information retrieval
- Trending/popularity rankings
- Hybrid recommendation engine
- Personalized user profiles with favorite movies